— Solution notes —

COMPUTER SCIENCE TRIPOS Part IB – 2024 – Paper 5

8 Introduction to Computer Architecture (swm11)

Memory hierarchy and caching; multicore processors.

(*a*) Why do multicore systems-on-chip use hierarchical caches rather than one large shared cache? [3 marks]

Answer: Caches are critical to reduce memory access latency to prevent cores from stalling waiting for data. One large shared cache would have a higher access latency due to the physical distances involved (length of wires, etc.). Instead a hierarchy of caches are used with smaller level-1 instruction and data caches physically close each to core, etc. These smaller physically closer caches will have much lower access latency.

For a multicore system, higher-levels of cache can be shared both to make better use of memory when not all cores are busy and to enable more efficient shared access.

OS Support

(*b*) Why are page tables hierarchical rather than flat? Illustrate your answer by considering the memory required to store the page table for:

- an application with a contiguous 6 MiB memory footprint starting at virtual address zero,
- running on a 32-bit processor, with a 32-bit physical address space and 4 KiB pages. [5 marks]

Answer: Page tables are stored in a hierarchical data structure to make them more compact. For the given 32b processor with 4 KiB pages you would have a 20b virtual page number to translate. A flat page table would require $2^{20} \times 4 = 4\ MiB$ of storage for a flat page table given that each page table entry is 4 bytes (20-bits of translation plus permissions, etc.). For a hierarchical page table we would split the virtual page number into two 10-bit values, with the most significant 10-bits being used to access the $2^{10} \times 4 = 4\ KiB$ root table. One leaf table will store 2^{10} entries each to 4 KiB pages. For 6 MiB of contiguous memory starting at virtual address zero we only need 6 MiB/4 KiB = $(6 \times 2^{20})/(4 \times 2^{10}) = 1.5 \times 2^{10}$ leaf entries, so 1.5 leaf tables each 4 KiB. We cannot have half a leaf table, so we end up with 2 with one half empty. So the total storage for the page tables is a 4 KiB root table and two 4 KiB leaf tables totaling 12 KiB.

OS Support

(*c*) What is a TLB and why do we need one per core? [3 marks]

Answer: The TLB is a Translation Look-aside Buffer that caches virtual to physical addresses translations. This translation is on the critical path in the pipeline so low latency is paramount. Thus, each core needs its own TLB.

Memory hierarchy and caching; multicore processors.

(*d*) What is the difference between inclusive, exclusive and non-inclusive non-exclusive (NINE) cache inclusion policies? [3 marks]

Answer: An inclusive cache retains all cache lines that are in the smaller caches closer to the processor core(s). Exclusive caches only contain cache lines that are not held by caches closer to the processor core(s). NINE caches hold onto cache lines even when they are sent to smaller caches nearer the processor core(s) but are allowed to evict cache lines if space is required.

Memory hierarchy and caching; multicore processors.

(*e*) In multicore system with private level-1 and level-2 caches and one shared level-3 cache, why might the level-2 cache be inclusive but the level-3 cache be NINE? [3 marks]

Answer: The level-2 cache is likely to be inclusive of the level-1 cache so that it can handle cache coherency on behalf of both levels of cache.

The level-3 cache is usually NINE as the least worst option. If it was inclusive then it would have to store all of the contents of the level-2 caches for every core, which is likely to make it very large giving limited scope to hold data that is not present in any level-2 cache. If the level-3 cache was exclusive then data in a level-2 cache for one core would not be present in the level-3, so if another core wanted the same data then it would missing the level-3 cache, which would be very inefficient for multithreaded applications working on the same dataset.

Memory hierarchy and caching; multicore processors.

(*f*) What is the difference between cache coherence and consistency? Give examples of two consistency models. [3 marks]

Answer: Cache coherence determines ordering of writes at a particular address (typically for a particular cache-lines worth of data). All processor cores will see updates in the same order.

Cache consistency is the ordering of updates across several cache lines. For example, sequential consistency means that all reads and writes are in-order. On the other hand a relaxed consistency model allows reordering of reads and writes across different cache lines.
